

Restaurant Management Analysis with DDD Approach

1. Ubiquitous Language:

- Customer
- Reservation
- Table
- Menu
- Order
- OrderItem
- Payment
- Staff
- Inventory

2. Bounded Contexts:

- Reservation Context → Manage reservations and tables
- Ordering Context → Manage orders and their status
- Billing Context → Manage invoices and payments
- Kitchen Context → Manage food preparation
- Inventory Context → Manage raw materials and stock

3. Aggregates and Entities:

Context	Aggregate Root	Entities
Ordering	Order	OrderItem
Reservation	Reservation	Table
Billing	Invoice	Payment
Kitchen	KitchenOrder	-
Inventory	StockItem	Supplier

4. Value Objects:

- Money (Amount + Currency)
- TableNumber
- MenuItemDescription

5. Domain Services:

- OrderPricingService → calculate order total with discounts
- ReservationService → check table availability

6. Repositories:

- IOrderRepository
- IReservationRepository
- IInvoiceRepository

7. Event Driven Design:

- OrderCreated → sent to Kitchen Context
- OrderCompleted → sent to Billing Context
- PaymentSucceeded → notify Reservation/Ordering to close order

8. Example C# Code (Order Aggregate):

```
public class Order {
    public Guid Id { get; private set; }
    public Guid CustomerId { get; private set; }
    public Guid TableId { get; private set; }
    private readonly List<OrderItem> _items = new();
    public IReadOnlyCollection<OrderItem> Items => _items.AsReadOnly();
    public OrderStatus Status { get; private set; }

    private Order() { } // For EF

    public Order(Guid customerId, Guid tableId) {
        Id = Guid.NewGuid();
        CustomerId = customerId;
        TableId = tableId;
        Status = OrderStatus.Pending;
    }

    public void AddItem(Guid productId, int quantity, decimal price) {
        if (Status != OrderStatus.Pending)
            throw new InvalidOperationException("Cannot add items after order is started.");

        _items.Add(new OrderItem(productId, quantity, price));
    }

    public decimal GetTotalAmount() => _items.Sum(i => i.Total);
    public void MarkInProgress() => Status = OrderStatus.InProgress;
    public void MarkCompleted() => Status = OrderStatus.Completed;
}
```